

lab05

Lucas rocha

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1    ✓ readr      2.2.0
✓ forcats    1.0.1    ✓ stringr    1.6.0
✓ ggplot2     4.0.3    ✓ tibble     3.3.1
✓ lubridate  1.9.5    ✓ tidyr      1.3.2
✓ purrr       1.2.2
— Conflicts — tidyverse_conflicts()
—
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
  conflicts to become errors
```

```
library(readr)
library(lubridate)
library(leaflet)
library(sf)
```

```
Linking to GEOS 3.14.1, GDAL 3.12.1, PROJ 9.7.1; sf_use_s2() is TRUE
```

Leitura e filtragem em blocos (aeronave N431WN)

```
tail_number_alvo <- "N431WN"
arquivo_voos <- "flights.csv.zip"

tabela_voos_bruta <- read_csv_chunked(
  file = arquivo_voos,
  callback = DataFrameCallback$new(function(chunk, index) {
    chunk %>% filter(TAIL_NUMBER == tail_number_alvo)
  }),
  chunk_size = 500000,
```

```

show_col_types = FALSE
)

```

Limpeza de dados, cálculo de datas/horários e ordenação

```

tabela_trajeto <- tabela_voos_bruta %>%
  filter(CANCELLED == 0, DIVERTED == 0) %>%
  mutate(
    DEPARTURE_TIME_STR = sprintf("%04d", as.integer(DEPARTURE_TIME)),
    HORA_PARTIDA = substr(DEPARTURE_TIME_STR, 1, 2),
    MINUTO_PARTIDA = substr(DEPARTURE_TIME_STR, 3, 4),
    DATA_HORA_PARTIDA = make_datetime(
      year = YEAR,
      month = MONTH,
      day = DAY,
      hour = as.numeric(HORA_PARTIDA),
      min = as.numeric(MINUTO_PARTIDA)
    )
  ) %>%
  arrange(DATA_HORA_PARTIDA) %>%
  mutate(
    DURACAO_HORAS = AIR_TIME / 60,
    VELOCIDADE_MEDIA_MPH = if_else(DURACAO_HORAS > 0, DISTANCE / DURACAO_HORAS,
NA_real_)
  )

```

Leitura de aeroportos e junção das coordenadas de Origem/Destino

```

tabela_aeroportos <- read_csv("airports.csv", show_col_types = FALSE) %>%
  select(IATA_CODE, AIRPORT, CITY, STATE, LATITUDE, LONGITUDE)

tabela_trajeto_geo <- tabela_trajeto %>%
  left_join(tabela_aeroportos, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(NOME_ORIGEM = AIRPORT, LAT_ORIGEM = LATITUDE, LONG_ORIGEM = LONGITUDE)
%>%
  left_join(tabela_aeroportos, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  rename(NOME_DESTINO = AIRPORT, LAT_DESTINO = LATITUDE, LONG_DESTINO =
LONGITUDE) %>%
  filter(!is.na(LAT_ORIGEM) & !is.na(LAT_DESTINO))

```

Transformação dos pontos em segmentos de linha geográficos (sf)

```
#
=====
# Chunk 5: Transformação dos pontos em segmentos de linha geográficos (sf)
#
=====
lista_geometrias <- vector("list", nrow(tabela_trajeto_geo))
for (i in seq_len(nrow(tabela_trajeto_geo))) {
  matriz_coordenadas <- matrix(
    c(
      tabela_trajeto_geo$LONG_ORIGEM[i], tabela_trajeto_geo$LAT_ORIGEM[i],
      tabela_trajeto_geo$LONG_DESTINO[i], tabela_trajeto_geo$LAT_DESTINO[i]
    ),
    ncol = 2, byrow = TRUE
  )
  lista_geometrias[[i]] <- st_linestring(matriz_coordenadas)
}
segmentos_rotas_sf <- st_sf(
  tabela_trajeto_geo,
  geometry = st_sfc(lista_geometrias, crs = 4326)
)
aeroportos_unicos <- bind_rows(
  tabela_trajeto_geo %>% select(IATA = ORIGIN_AIRPORT, NOME = NOME_ORIGEM, LAT
= LAT_ORIGEM, LONG = LONG_ORIGEM),
  tabela_trajeto_geo %>% select(IATA = DESTINATION_AIRPORT, NOME = NOME_DESTINO,
LAT = LAT_DESTINO, LONG = LONG_DESTINO)
) %>% distinct(IATA, .keep_all = TRUE)
```

Geração do mapa interativo Leaflet

```
paleta_velocidade <- colorNumeric(
  palette = "YlOrRd",
  domain = segmentos_rotas_sf$VELOCIDADE_MEDIA_MPH,
  na.color = "#808080"
)

mapa_trajeto <- leaflet() %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircleMarkers(
    data = aeroportos_unicos,
    lng = ~LONG, lat = ~LAT,
    radius = 5, color = "#2C3E50", fillOpacity = 0.8,
    popup = ~paste0("<b>Aeroporto: </b>", NOME, " (", IATA, ")")
  ) %>%
```

```

addPolylines(
  data = segmentos_rotas_sf,
  color = ~paleta_velocidade(VELOCIDADE_MEDIA_MPH),
  weight = ~pmax(2, VELOCIDADE_MEDIA_MPH / 120),
  opacity = 0.6,
  popup = ~paste0(
    "<b>Trajeto: </b>", ORIGIN_AIRPORT, " → ", DESTINATION_AIRPORT, "<br>",
    "<b>Data Partida: </b>", DATA_HORA_PARTIDA, "<br>",
    "<b>Distância: </b>", DISTANCE, " milhas<br>",
    "<b>Velocidade Média: </b>", round(VELOCIDADE_MEDIA_MPH, 1), " mph"
  )
) %>%
addLegend(
  pal = paleta_velocidade,
  values = segmentos_rotas_sf$VELOCIDADE_MEDIA_MPH,
  title = "Velocidade Média (mph)",
  position = "bottomright"
)

```

Função analisa_aeronave Modularizada

```

analisa_aeronave <- function(tail_number,
                             arquivo = file.path("dados", "flights.csv.zip"),
                             arquivo_airports = file.path("dados", "airports.csv")) {

  tabela_voos_bruta <- read_csv_chunked(
    file = arquivo,
    callback = DataFrameCallback$new(function(chunk, index) {
      chunk %>% filter(TAIL_NUMBER == tail_number)
    }),
    chunk_size = 500000,
    show_col_types = FALSE
  )

  tabela_trajeto <- tabela_voos_bruta %>%
    filter(CANCELLED == 0, DIVERTED == 0) %>%
    mutate(
      DEPARTURE_TIME_STR = sprintf("%04d", as.integer(DEPARTURE_TIME)),
      HORA_PARTIDA = substr(DEPARTURE_TIME_STR, 1, 2),
      MINUTO_PARTIDA = substr(DEPARTURE_TIME_STR, 3, 4),
      DATA_HORA_PARTIDA = make_datetime(
        year = YEAR, month = MONTH, day = DAY,
        hour = as.numeric(HORA_PARTIDA), min = as.numeric(MINUTO_PARTIDA)
      )
    )
}

```

```

    )
  ) %>%
  arrange(DATA_HORA_PARTIDA) %>%
  mutate(
    DURACAO_HORAS = AIR_TIME / 60,
    VELOCIDADE_MEDIA_MPH = if_else(DURACAO_HORAS > 0, DISTANCE / DURACAO_HORAS,
NA_real_)
  )

tabela_aeroportos <- read_csv(arquivo_airports, show_col_types = FALSE) %>%
  select(IATA_CODE, AIRPORT, CITY, STATE, LATITUDE, LONGITUDE)

tabela_trajeto_geo <- tabela_trajeto %>%
  left_join(tabela_aeroportos, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(NOME_ORIGEM = AIRPORT, LAT_ORIGEM = LATITUDE, LONG_ORIGEM = LONGITUDE)
%>%
  left_join(tabela_aeroportos, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  rename(NOME_DESTINO = AIRPORT, LAT_DESTINO = LATITUDE, LONG_DESTINO =
LONGITUDE) %>%
  filter(!is.na(LAT_ORIGEM) & !is.na(LAT_DESTINO))

lista_geometrias <- vector("list", nrow(tabela_trajeto_geo))
for (i in seq_len(nrow(tabela_trajeto_geo))) {
  matriz_coordenadas <- matrix(
    c(
      tabela_trajeto_geo$LONG_ORIGEM[i], tabela_trajeto_geo$LAT_ORIGEM[i],
      tabela_trajeto_geo$LONG_DESTINO[i], tabela_trajeto_geo$LAT_DESTINO[i]
    ),
    ncol = 2, byrow = TRUE
  )
  lista_geometrias[[i]] <- st_linestring(matriz_coordenadas)
}

segmentos_rotas_sf <- st_sf(
  tabela_trajeto_geo,
  geometry = st_sfc(lista_geometrias, crs = 4326)
)

aeroportos_unicos <- bind_rows(
  tabela_trajeto_geo %>% select(IATA = ORIGIN_AIRPORT, NOME = NOME_ORIGEM, LAT
= LAT_ORIGEM, LONG = LONG_ORIGEM),
  tabela_trajeto_geo %>% select(IATA = DESTINATION_AIRPORT, NOME = NOME_DESTINO,
LAT = LAT_DESTINO, LONG = LONG_DESTINO)

```

```

) %>% distinct(IATA, .keep_all = TRUE)

paleta_velocidade <- colorNumeric(
  palette = "YlOrRd",
  domain = segmentos_rotas_sf$VELOCIDADE_MEDIA_MPH,
  na.color = "#808080"
)

mapa <- leaflet() %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircleMarkers(
    data = aeroportos_unicos,
    lng = ~LONG, lat = ~LAT,
    radius = 5, color = "#2C3E50", fillOpacity = 0.8,
    popup = ~paste0("<b>Aeroporto: </b>", NOME, " (", IATA, ")")
  ) %>%
  addPolylines(
    data = segmentos_rotas_sf,
    color = ~paleta_velocidade(VELOCIDADE_MEDIA_MPH),
    weight = ~pmax(2, VELOCIDADE_MEDIA_MPH / 120),
    opacity = 0.6,
    popup = ~paste0(
      "<b>Etapa: </b>", row_number(), "<br>",
      "<b>Trajeto: </b>", ORIGIN_AIRPORT, " → ", DESTINATION_AIRPORT, "<br>",
      "<b>Data Partida: </b>", DATA_HORA_PARTIDA, "<br>",
      "<b>Distância: </b>", DISTANCE, " milhas<br>",
      "<b>Velocidade Média: </b>", round(VELOCIDADE_MEDIA_MPH, 1), " mph"
    )
  ) %>%
  addLegend(
    pal = paleta_velocidade,
    values = segmentos_rotas_sf$VELOCIDADE_MEDIA_MPH,
    title = "Velocidade Média (mph)",
    position = "bottomright"
  )

return(list(
  tabela_trajeto = tabela_trajeto,
  mapa_trajeto = mapa
))
}

```

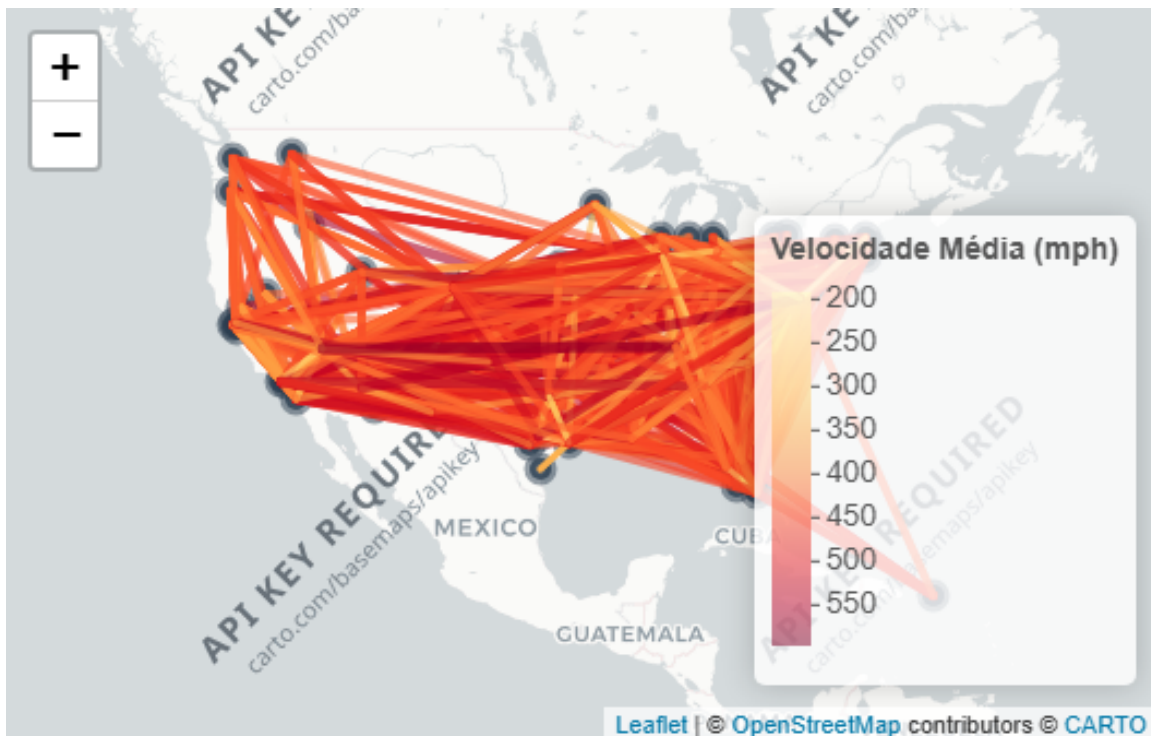
Visualizar as primeiras linhas da tabela ordenada

```
head(tabela_trajeto)
```

```
# A tibble: 6 × 37
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER ORIGIN_AIRPORT
  <dbl> <dbl> <dbl>         <dbl> <chr>         <dbl> <chr>         <chr>
1  2015     1     1             4 WN             1762 N431WN      DCA
2  2015     1     1             4 WN             3564 N431WN      HOU
3  2015     1     1             4 WN             3564 N431WN      BOS
4  2015     1     1             4 WN             3564 N431WN      BWI
5  2015     1     2             5 WN             1793 N431WN      MCI
6  2015     1     2             5 WN             1793 N431WN      LAX
# i 29 more variables: DESTINATION_AIRPORT <chr>, SCHEDULED_DEPARTURE <chr>,
# DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>, TAXI_OUT <dbl>,
# WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>, AIR_TIME <dbl>,
# DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>, SCHEDULED_ARRIVAL <chr>,
# ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>, DIVERTED <dbl>, CANCELLED <dbl>,
# CANCELLATION_REASON <chr>, AIR_SYSTEM_DELAY <dbl>, SECURITY_DELAY <dbl>,
# AIRLINE_DELAY <dbl>, LATE_AIRCRAFT_DELAY <dbl>, WEATHER_DELAY <dbl>, ...
```

Exibição do Mapa Final

```
mapa_trajeto
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-12 02:16:36 -03"
```